

Hotel Reservation System

Project Documentation



Course Code: CSC 2032

Course Name: Object Oriented Programming

Student 1 Index No: AS20240358

Student 2 Index No: AS20240441

Submitted Date: 2026.07.26

Table of Content

1. Introduction	2
2. Problem Statement	2
3. Objectives	3
4. Scope	3
5. Definitions, Acronyms and Abbreviations.....	4
6. Requirement Analysis.....	5
7. Technologies Used	7
8. System Architecture.....	8
9. Use Case Diagram	9
10. Sequence Diagram	10
11. Class Diagram	11
.....	11
12. Class Documentation.....	12
Models.....	13
Repositories	14
Services	14
Controllers.....	15
13. ER Diagram	16
14. Database Design	17
Collection Definitions	17
15. Backend Architecture and Execution Workflow	19
16. UI Documentation	21
17. Challenges	24
18. Testing	25
19. Future Enhancements	27
20. Conclusion	27
21. References.....	28

1. Introduction

The Hotel Reservation System is a web application that provides room browsing, reservation management, customer and admin workflows, messaging, and payment tracking. It is intended to help a hotel manage room availability, customer booking activity, and support communications while giving visitors an intuitive way to search and book rooms online.

Hotel management traditionally relies on spreadsheets, phone calls, and manual tracking of bookings and guests. This manual approach is hard to maintain. Modern hotel operations benefit from an integrated reservation platform where availability, reservations, guest profiles, and communication are centralized. This project is built as a full-stack solution with a Spring Boot backend and a React frontend. The backend exposes REST APIs for rooms, reservations, customers, payments, admin user management, dashboard analytics, and guest messaging. The frontend consumes these APIs and presents hotel operations through an interactive user interface

2. Problem Statement

Many small hotels and guesthouses still manage room bookings and guest information using disconnected tools. The problems include:

- Inconsistent room availability data
- Limited visibility of current reservations
- Difficult check-in/check-out processes
- Inefficient customer support communications
- Lack of centralized payment and reservation tracking
- Poor access control across customer and admin functions

There is a need for a centralized hotel reservation system that supports authenticated users, administrator controls, room search and booking, payment updates, and guest messaging.

3. Objectives

The primary objectives of the system are to:

- automate room booking and reservation lifecycle management
- provide secure user authentication and role-based access control
- support room inventory visualization and availability filtering
- manage reservation status transitions such as Reserved, Checked In, Checked Out, and Cancelled
- support customer and admin account management
- enable guest messaging and administrative reply handling
- provide dashboard-based operational summaries for administrators
- support email notifications for booking and inquiry events

4. Scope

The system supports the following features:

- customer registration and login
- admin authentication and admin account setup
- browsing and filtering rooms by availability and date range
- creating, updating, and deleting room records
- creating reservations with date validation and overlap checks
- updating payment status and managing reservation lifecycle state
- room status updates during check-in and check-out operations
- customer profile management
- guest message submission, reading, replying, and deletion

- admin dashboard summaries for rooms, customers, reservations, and occupancy
- email notifications for booking confirmation, cancellation, and guest messages

The current implementation does not include:

- integration with a real external payment gateway
- native mobile applications
- multi-currency or international payment settlement features
- IoT-based room access integration
- real-time chat or live customer support channels
- advanced analytics or AI-based pricing optimization

5. Definitions, Acronyms and Abbreviations

- SRS: Software Requirement Specification
- UI: User Interface
- JWT: JSON Web Token
- CRUD: Create, Read, Update, Delete
- Admin: System administrator with management privileges
- Customer: End user who books rooms

6. Requirement Analysis

ID	Requirement
FR-1	The system shall allow a customer to register using full name, email, password, NIC/passport, phone, and address.
FR-2	The system shall allow a customer or admin to authenticate using email/username and password.
FR-3	The system shall issue a JWT token after successful authentication for protected API access.
FR-4	The system shall enforce role-based access so that customers can manage their own reservations and messages, while admins can manage inventory and operational data.
FR-5	The system shall allow admins to create, update, delete, and view room records.
FR-6	The system shall allow customers to search rooms by check-in and check-out dates.
FR-7	The system shall prevent overlapping reservations for the same room when the requested dates conflict with an existing reservation.
FR-8	The system shall validate that the check-out date is after the check-in date before a reservation is created or updated.
FR-9	The system shall create a reservation with status Reserved and payment status Pending by default.
FR-10	The system shall support reservation status transitions to Checked In, Checked Out, and Cancelled.
FR-11	The system shall update room status to Reserved, Occupied, or Available as reservation lifecycle events occur.
FR-12	The system shall allow admins update reservation payment statu

FR-13	The system shall allow customers to view their own reservation history.
FR-14	The system shall allow admins to view all reservations and customer records.
FR-15	The system shall allow guest users to submit support or inquiry messages.
FR-16	The system shall allow admins to mark guest messages as read and reply to them.
FR-17	The system shall support email notification for booking confirmation, cancellation.
FR-18	The system shall provide an admin dashboard with summary information such as total rooms, available rooms, reserved room occupied rooms, customer count, and active reservations.

Non Functional Requirements

Category	Requirement
Usability	The UI shall provide clear navigation for guests and administrators, including room browsing, booking, and administrative management panels.
Reliability	The system shall handle invalid input and missing configuration gracefully and continue operating without crashing.
Data Integrity	The system shall validate reservation dates and prevent conflicting room reservations to preserve logical consistency.
Security	The system shall use BCrypt password hashing, JWT-based authentication, and role-based endpoint authorization.
Maintainability	The backend shall keep domain logic separated from controllers through dedicated service classes.

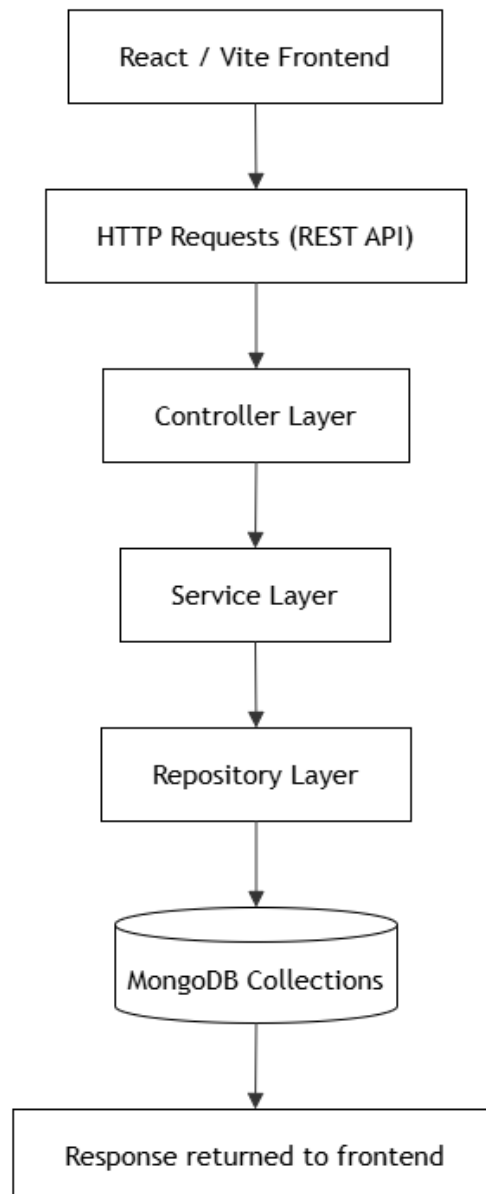
Performance	The system shall respond to room listing, availability checks, and dashboard summary requests efficiently for a single-module web application.
-------------	--

7. Technologies Used

Category	Technology
Language	Java 17
Backend Framework	Spring Boot 3.5.16
Web Layer	Spring Web / REST Controllers
Security	Spring Security, JWT, BCrypt
Persistence	Spring Data MongoDB
Validation	Spring Validation
Email	Spring Mail
Frontend	React + Vite
Styling	Tailwind CSS
Build Tools	Maven, npm

8. System Architecture

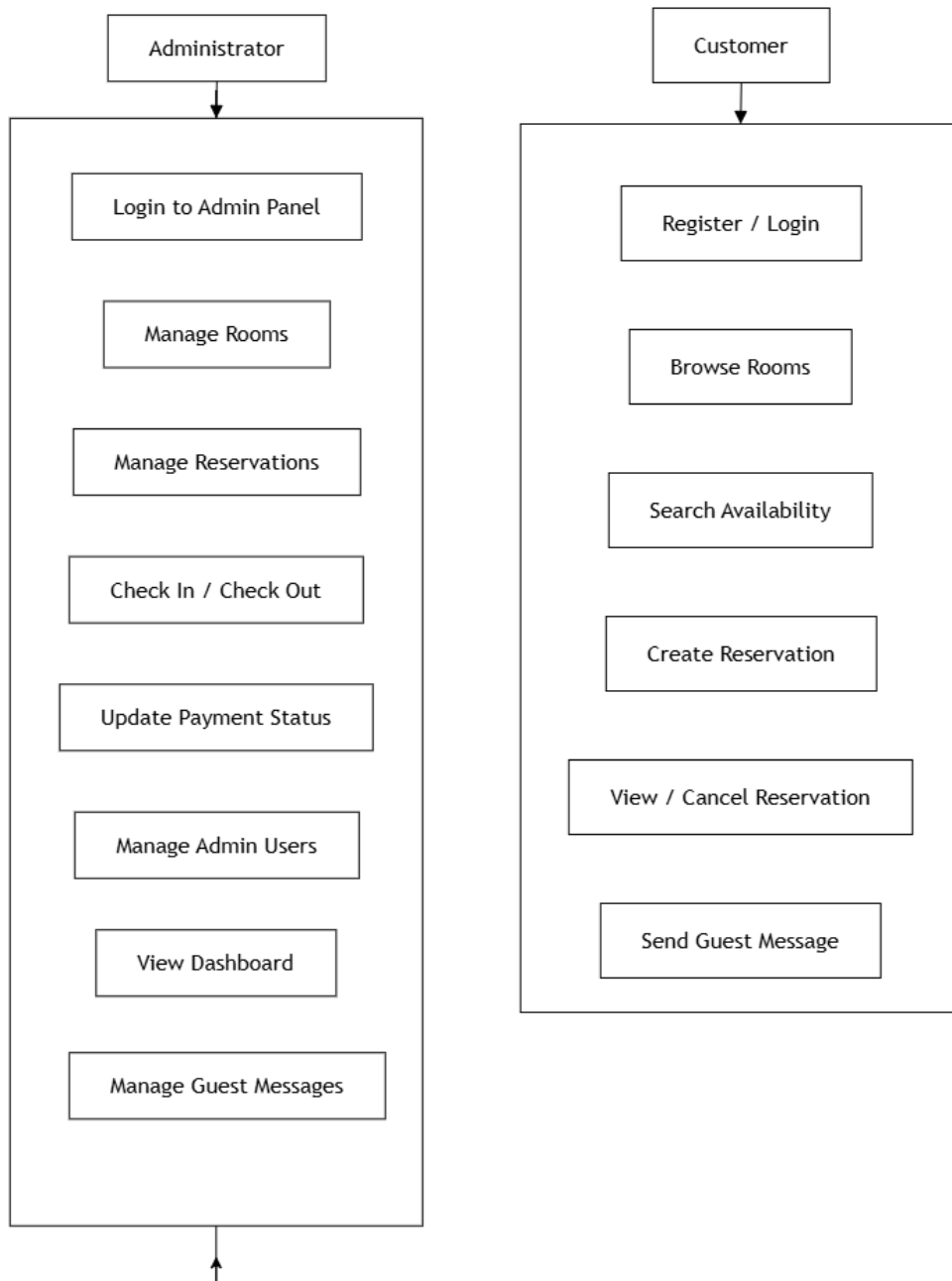
The system is implemented as a layered application with the following processing flow:



The controllers expose REST endpoints for authentication, room management, reservations, payments, customers, guest messages, and dashboard summaries. The services contain business logic such as room availability validation, reservation lifecycle rules, payment update operations, and email dispatch. The repositories provide persistence operations over MongoDB documents.

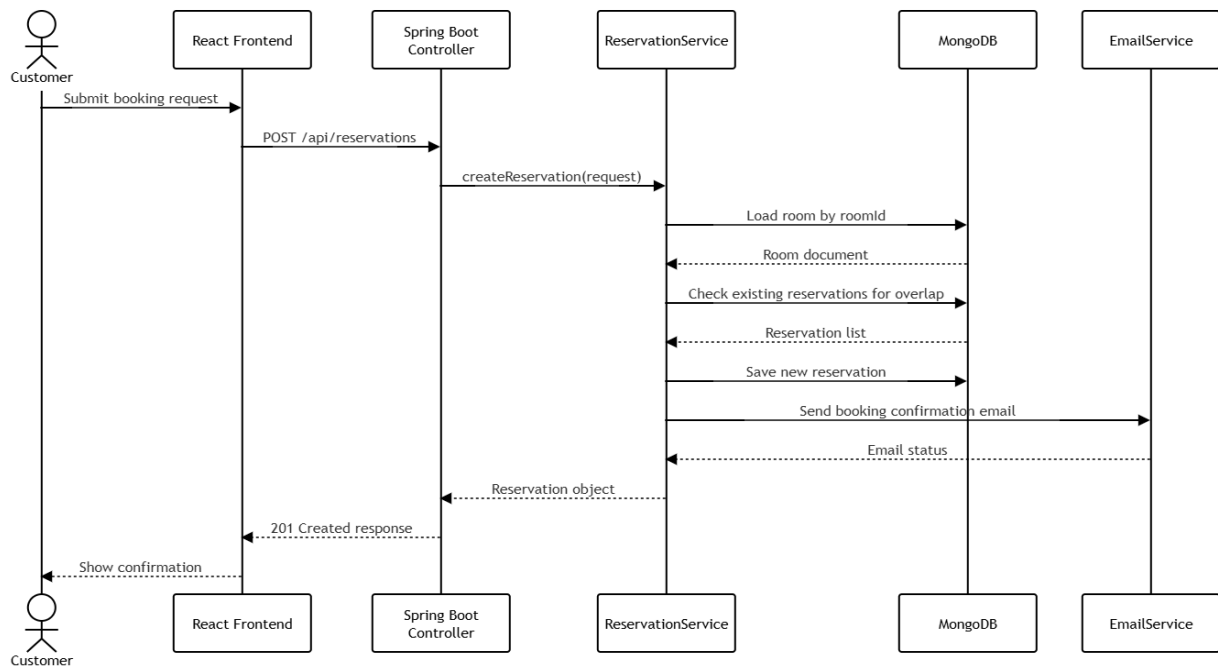
9. Use Case Diagram

The system consists of two roles, User and Administrator:

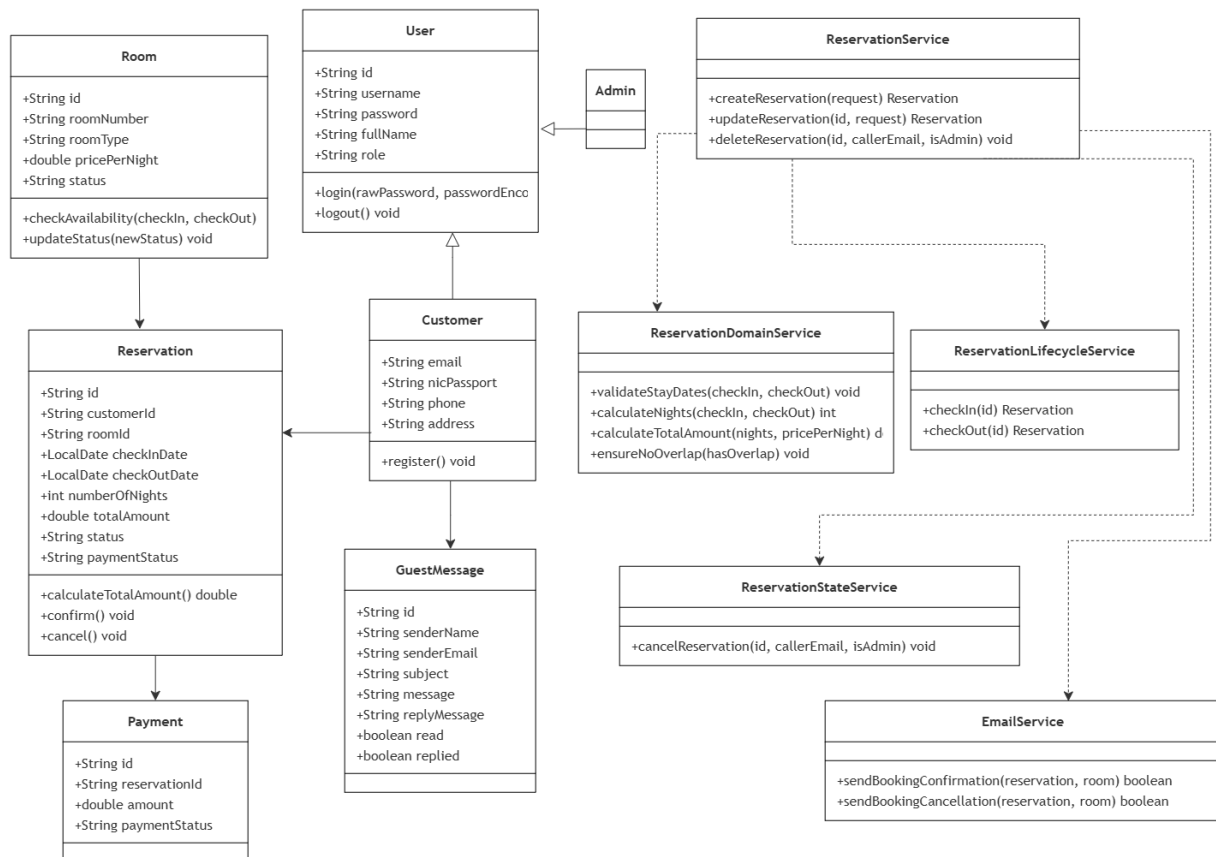


10. Sequence Diagram

The following sequence describes the reservation booking flow from the customer perspective.



11. Class Diagram



The implementation uses a domain-oriented class structure with inheritance and service composition.

12. Class Documentation

Main Domain Classes

Class	Core Attributes	Key Methods	Relationships
User	id, username, password, fullName, role	login(), logout()	Base class for Customer and Admin
Customer	email, nicPassport, phone, address	register()	Inherits User; creates reservations and messages
Admin	-	manageRooms(), manageReservations(), updatePaymentStatus()	Inherits User
Room	id, roomNumber, roomType, pricePerNight, status, description, imageUrl, roomSize, bedType, capacity, amenities	checkAvailability(), updateStatus()	Used by Reservation
Reservation	customerId, customerName, customerEmail, roomId, checkInDate, checkOutDate, numberOfGuests, numberOfNights, totalAmount, status, paymentStatus, actualCheckIn, actualCheckOut	calculateTotalAmount(), confirm(), cancel()	Associated with Room, Customer, Payment
Payment	reservationId, amount, paymentDate, paymentMethod, paymentStatus	processPayment(), refund()	Related to Reservation

GuestMessage	senderName, senderEmail, subject, message, source, replyMessage, read, replied, createdAt, updatedAt	send(), reply()	Sent by Customer; reviewed by Admin
--------------	--	-----------------	-------------------------------------

Supporting Services Classes

Class	Responsibility
ReservationService	Orchestrates reservation creation, update, deletion, and lifecycle requests
ReservationDomainService	Validates dates, calculates nights and totals, and enforces overlap restrictions
ReservationLifecycleService	Performs check-in and check-out operations
ReservationStateService	Handles reservation cancellation and room availability reset
RoomService	Manages room read/write operations and availability filtering
AuthService	Handles registration, login support, admin setup, and admin user management
PaymentService	Creates and updates payments
GuestMessageService	Creates, reads, replies to, and deletes guest messages
DashboardService	Produces summary statistics for admin dashboards
EmailService	Sends booking and guest message notification emails

Models

- User: base identity model with username, password, role, and authentication helpers.

- Customer: extends User and stores customer-specific fields such as email, NIC/passport, phone, and address.
- Admin: extends User and provides placeholder methods for room and reservation management actions.
- Room: represents a room entity with pricing, room type, amenities, and status.
- Reservation: encapsulates a booking and includes check-in/check-out dates, nights, total amount, and lifecycle status.
- Payment: stores payment-related information for a reservation.
- GuestMessage: stores guest inquiry messages and admin replies with email tracking metadata.

Repositories

- UserRepository: persistence operations for users.
- CustomerRepository: persistence operations for customers.
- RoomRepository: persistence operations for rooms, room status lookup, and counting.
- ReservationRepository: persistence operations for reservations, availability overlap queries, and status-based queries.
- PaymentRepository: persistence operations for payment records.
- GuestMessageRepository: persistence operations for guest messages.

Services

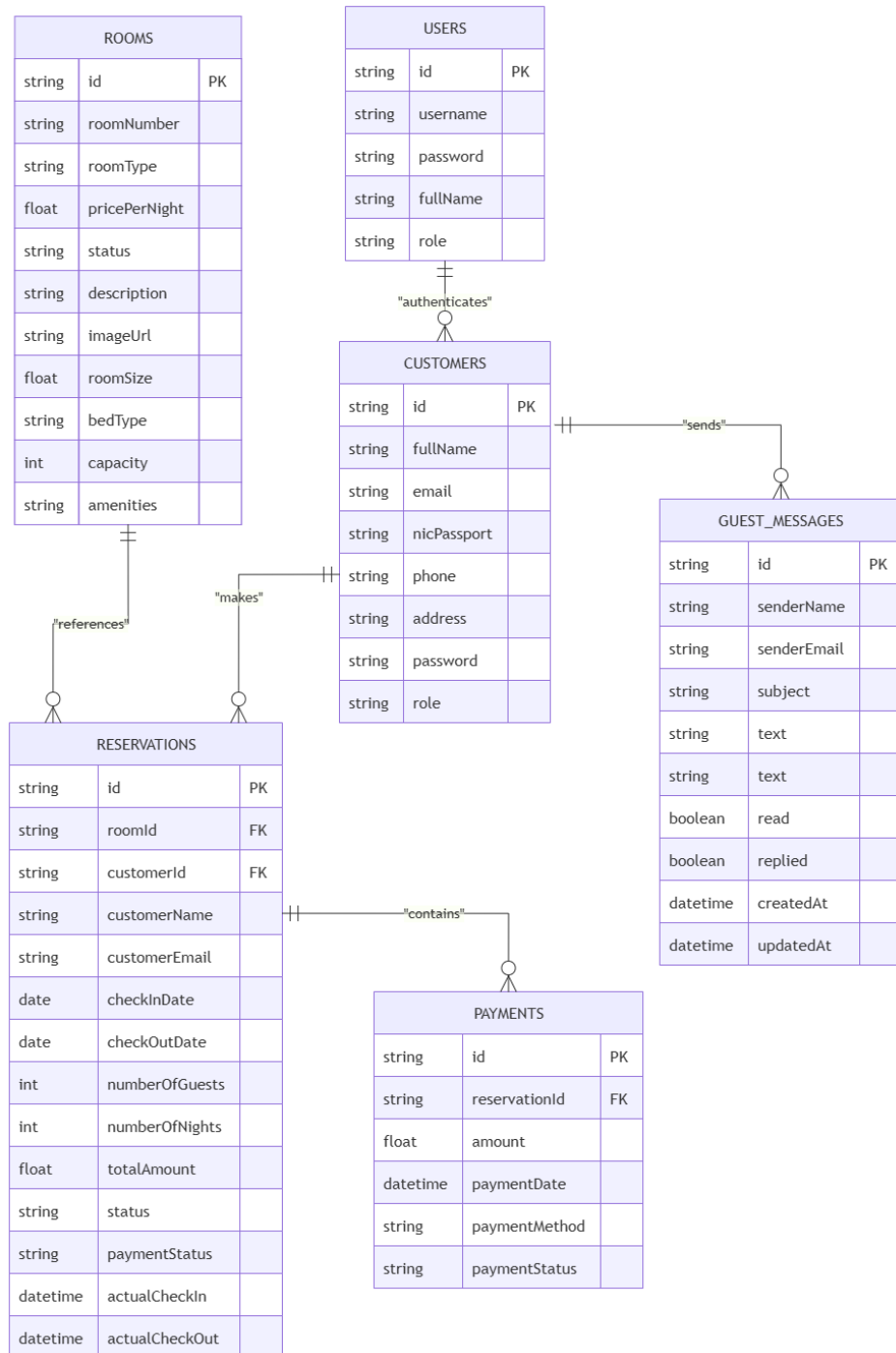
- AuthService: handles customer registration, default admin setup, admin password setup, admin creation, and deletion.
- RoomService: manages room creation, update, delete, retrieval, and availability filtering.

- **ReservationService**: coordinates reservation creation, update, deletion, and lifecycle methods.
- **ReservationDomainService**: validates stay dates, computes total amount, and enforces overlap rules.
- **ReservationLifecycleService**: processes check-in and check-out transitions.
- **ReservationStateService**: cancels reservations and restores room availability when appropriate.
- **PaymentService**: creates and updates payment records.
- **GuestMessageService**: creates, retrieves, marks as read, replies to, and deletes guest messages.
- **DashboardService**: retrieves summary metrics for the admin dashboard.
- **EmailService**: sends HTML emails for booking confirmation, cancellation, and guest messages.

Controllers

- **AuthController**: exposes login, registration, and admin-password-setup endpoints.
- **RoomController**: exposes room CRUD and availability endpoints.
- **ReservationController**: exposes reservation CRUD, payment update, check-in, check-out, and customer-specific queries.
- **PaymentController**: exposes payment creation and update endpoints.
- **CustomerController**: exposes customer retrieval, creation, update, and deletion endpoints.
- **GuestMessageController**: exposes guest message submission, retrieval, reply, read, and deletion endpoints.
- **DashboardController**: exposes dashboard summary endpoint.
- **AdminController**: exposes admin-user management endpoints.

13. ER Diagram



14. Database Design

The persistence layer uses MongoDB collections. The following collections are used by the current implementation:

Table / Collection Name	Purpose
users	Stores authentication and user identity records for admins and system users
customers	Stores customer profile information
rooms	Stores hotel room inventory and room status
reservations	Stores booking records and lifecycle state
payments	Stores payment records related to reservations
guest_messages	Stores customer inquiry and admin reply data

Collection Definitions

Collection	Primary Key	Foreign Keys / References	Key Fields
users	id	None	username, password, fullName, role
customers	id	None	fullName, email, nicPassport, phone, address, password, role
rooms	id	None	roomNumber, roomType, pricePerNight,

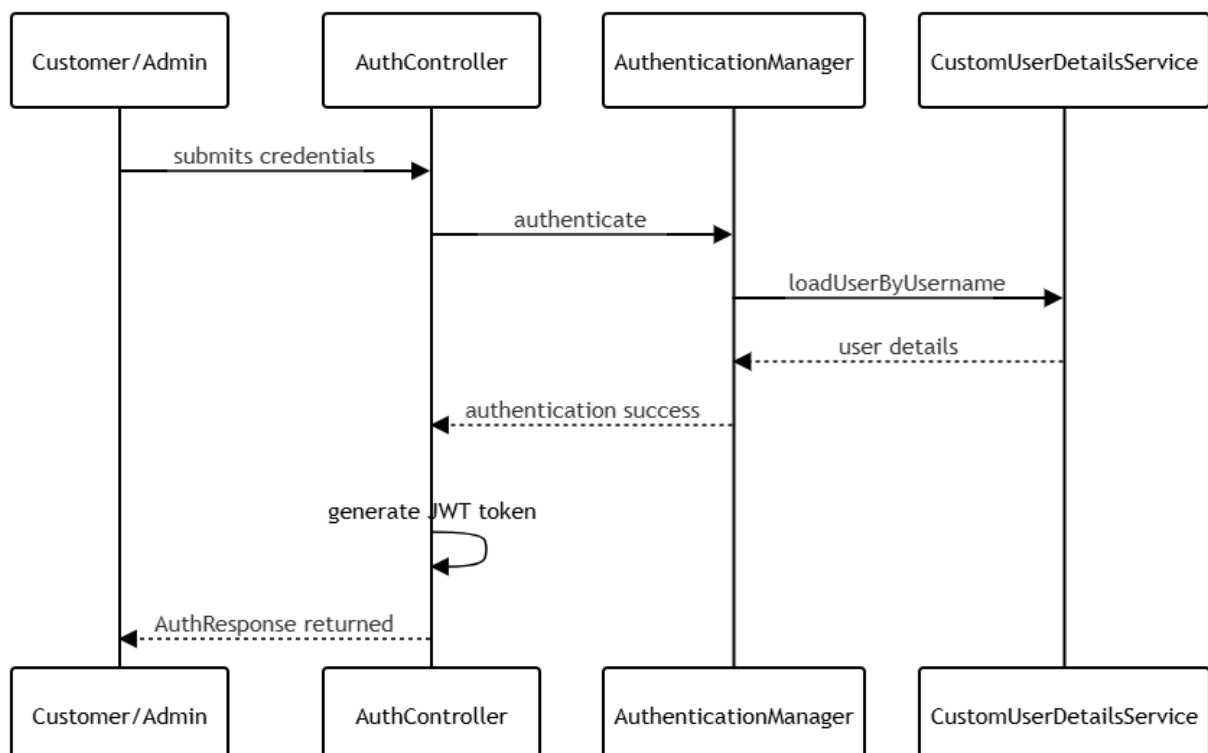
			status, description, imageUrl, roomSize, bedType, capacity, amenities
reservations	id	roomId, customerId	customerName, customerEmail, checkInDate, checkOutDate, numberOfGuests, numberOfNights, totalAmount, status, paymentStatus, actualCheckIn, actualCheckOut
payments	id	reservationId	amount, paymentDate, paymentMethod, paymentStatus
guest_messages	id	None	senderName, senderEmail, subject, message, replyMessage, read, replied, createdAt, updatedAt

15. Backend Architecture and Execution Workflow

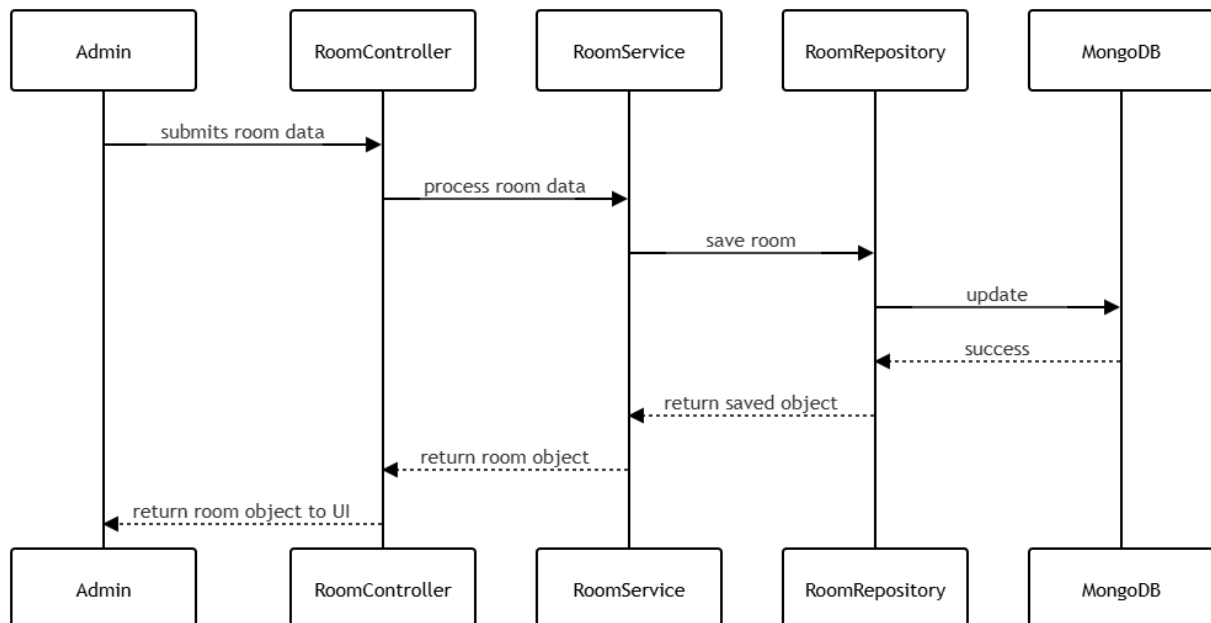
Layered Architecture

1. The frontend sends HTTP requests to the REST API.
2. Controllers receive requests and delegate to services.
3. Services apply business rules and interact with repositories.
4. Repositories persist to MongoDB collections.
5. Responses are returned to the UI for rendering.

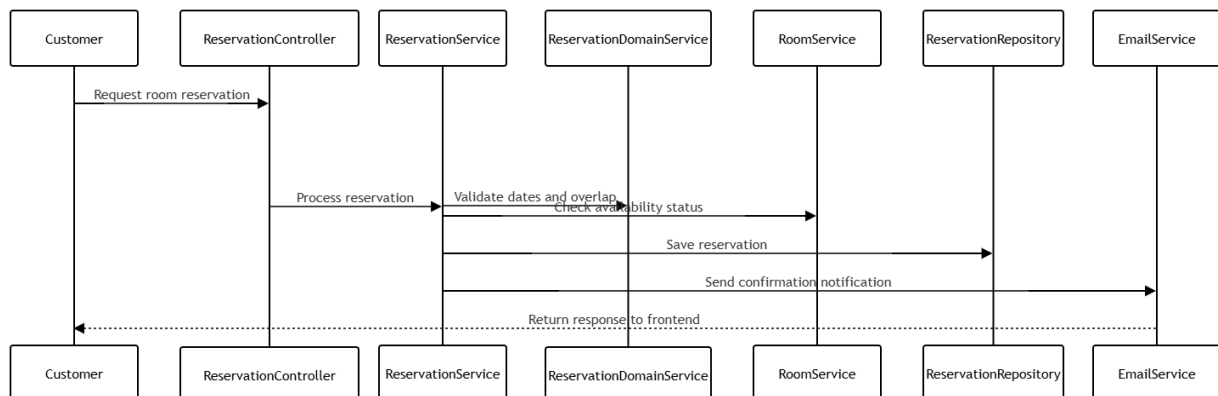
Authentication Flow



Room Inventory Management Flow

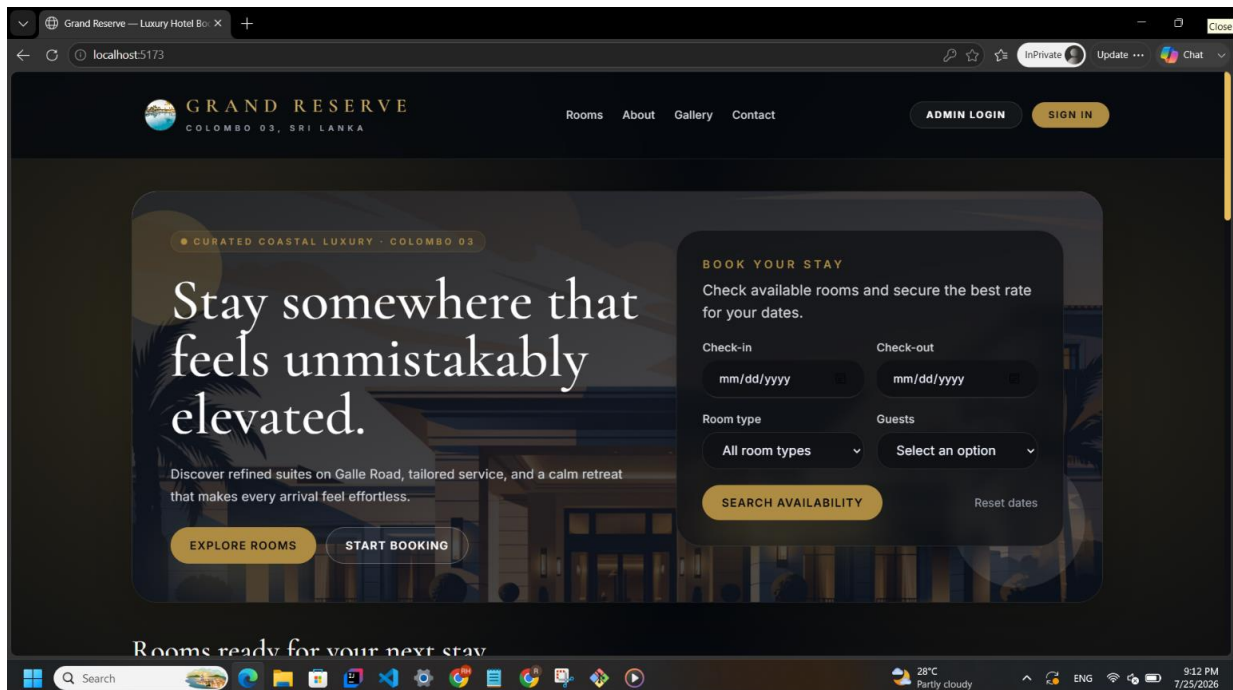


Reservation and Checkout Flow

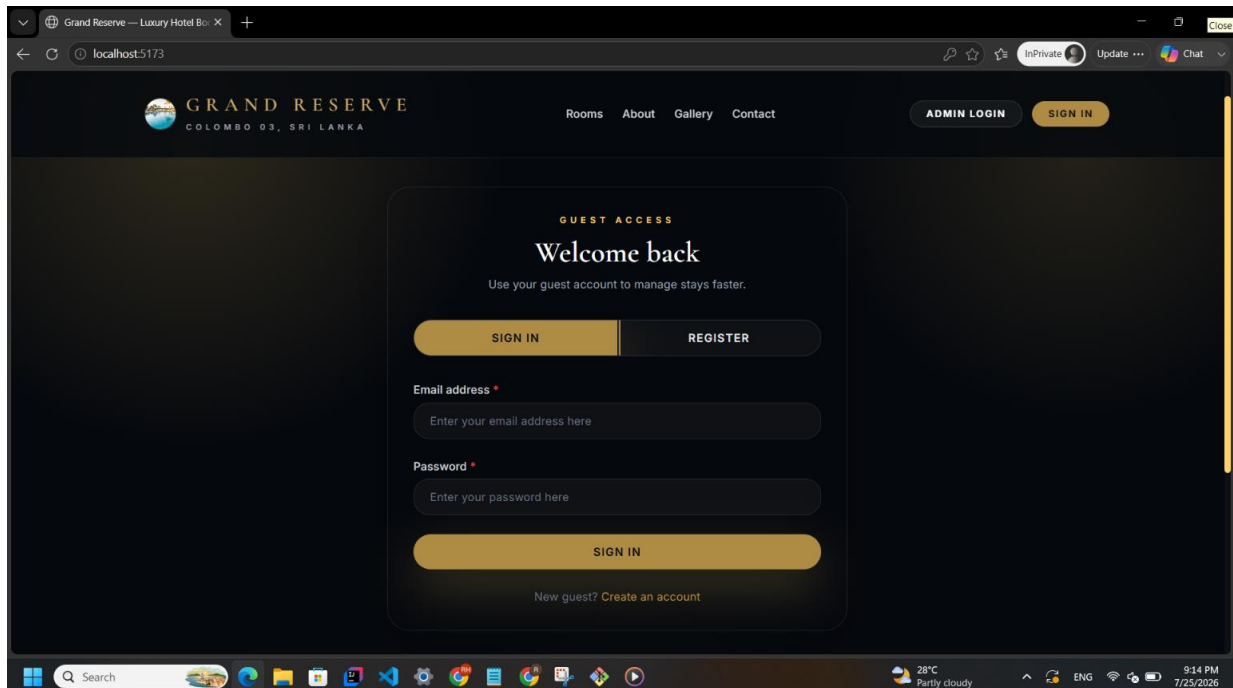


16. UI Documentation

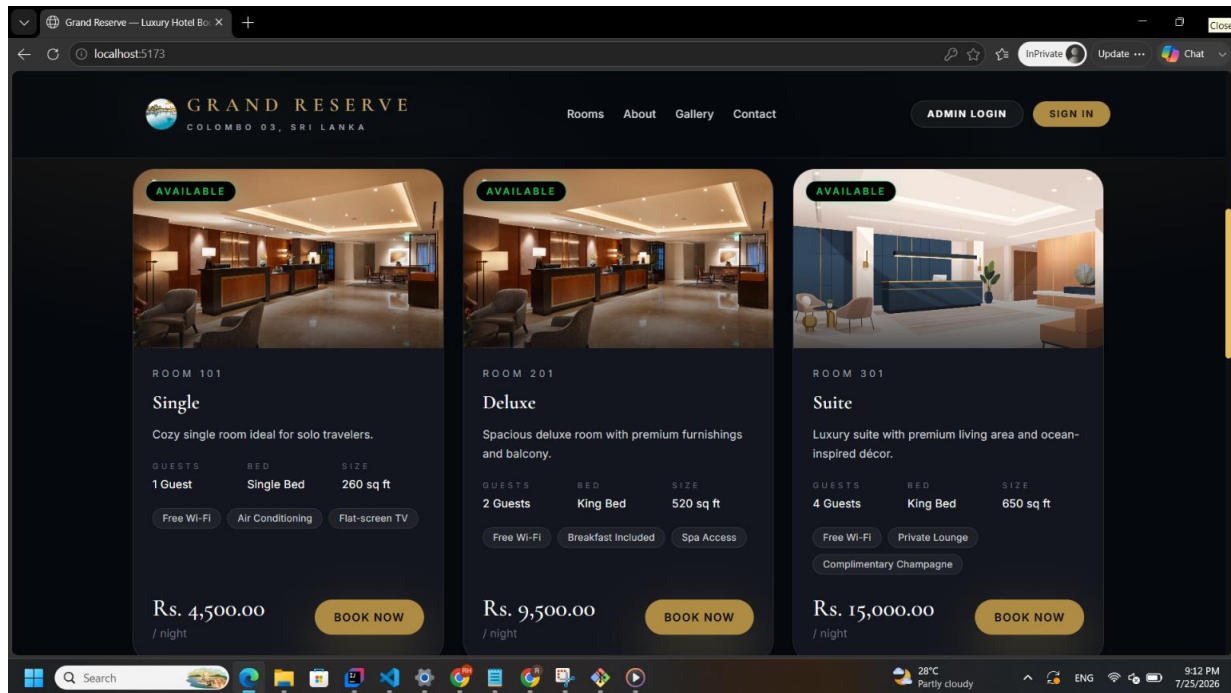
Home Page



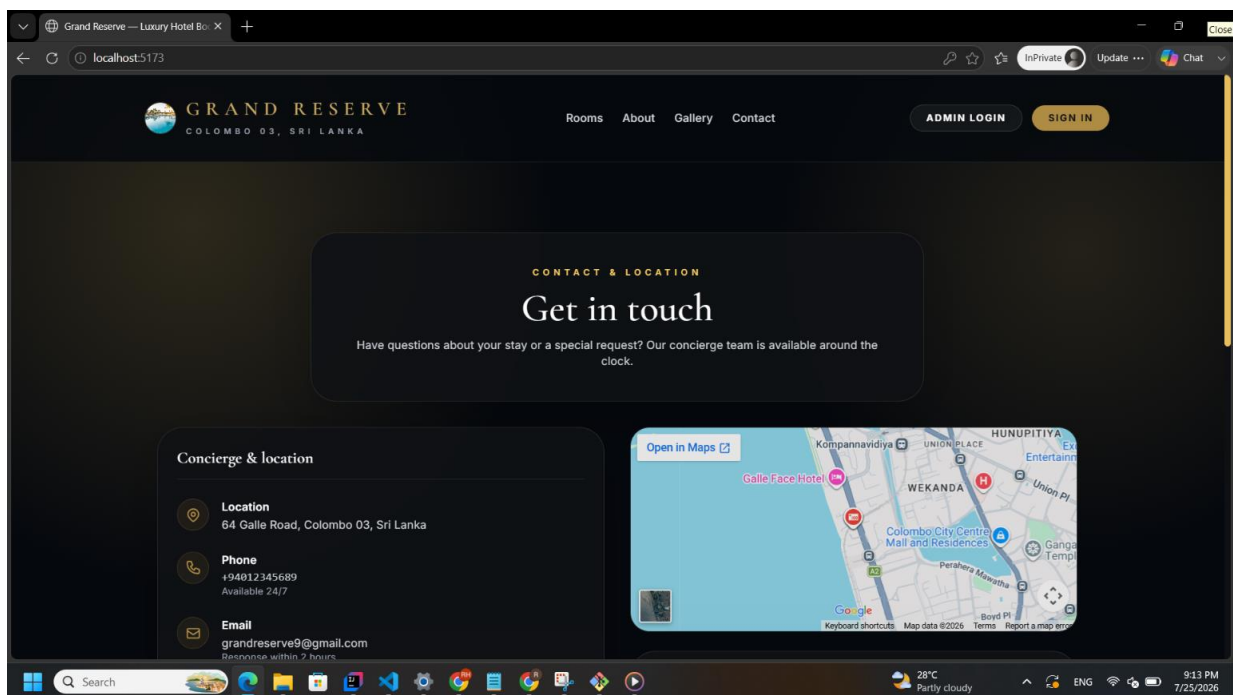
Login Page



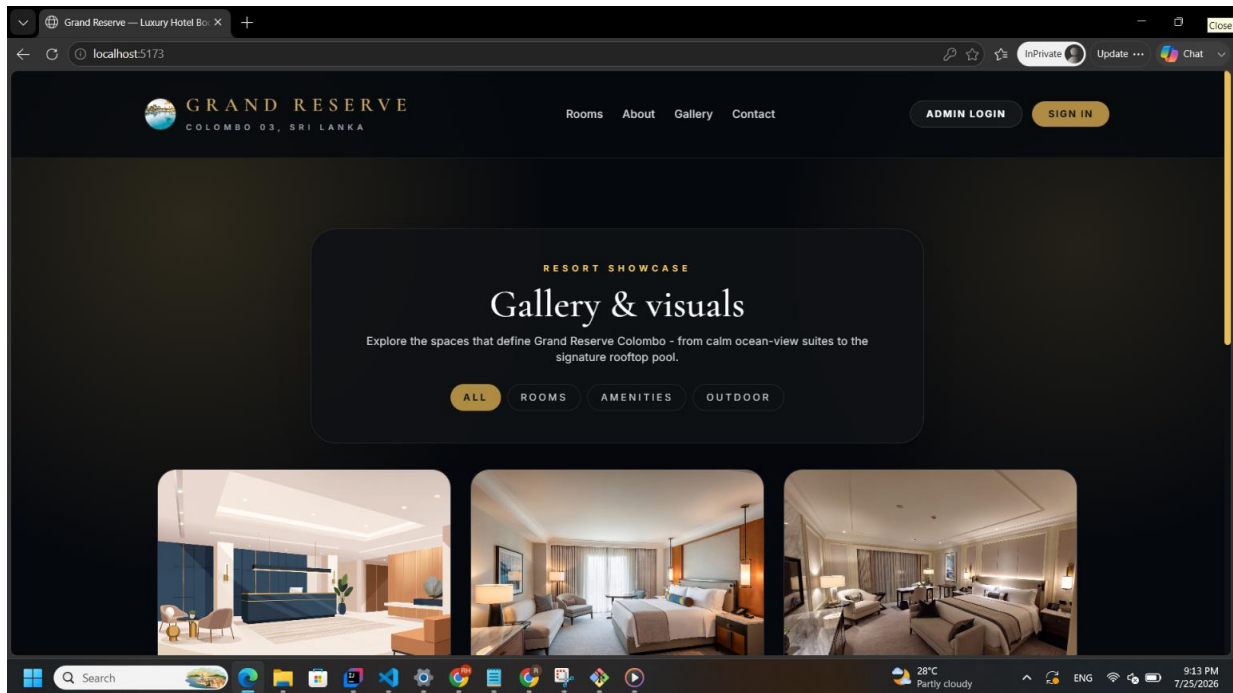
Find Rooms Section



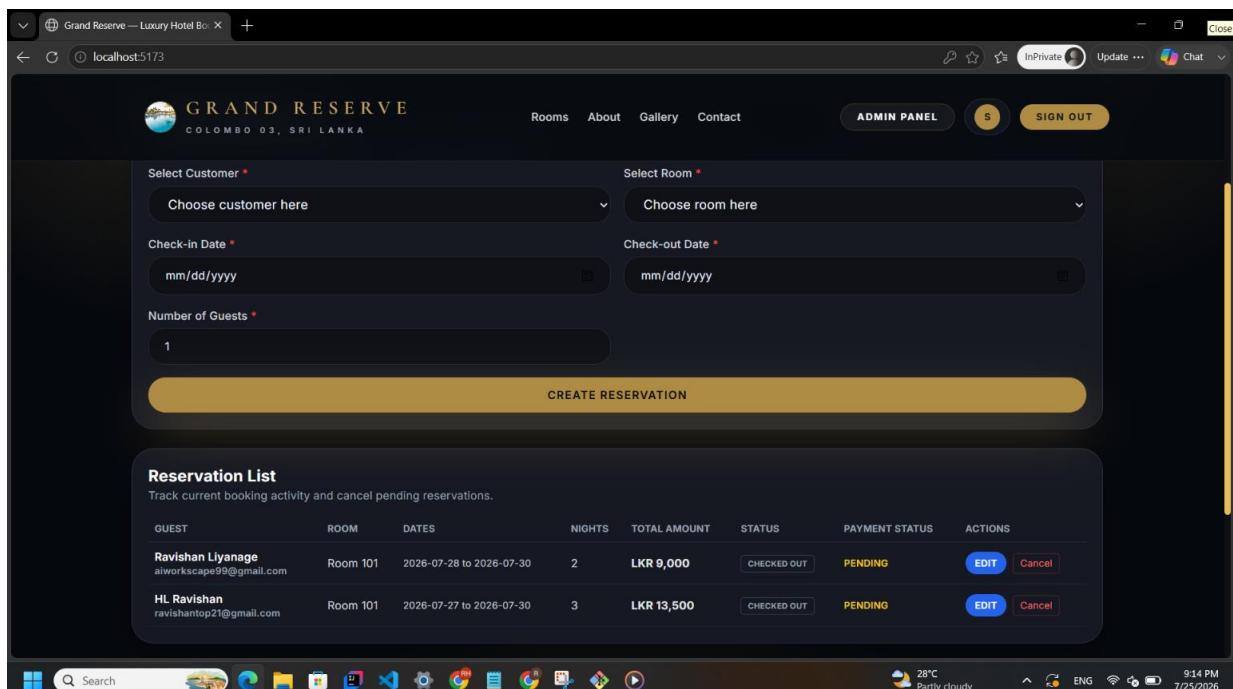
Contact Section



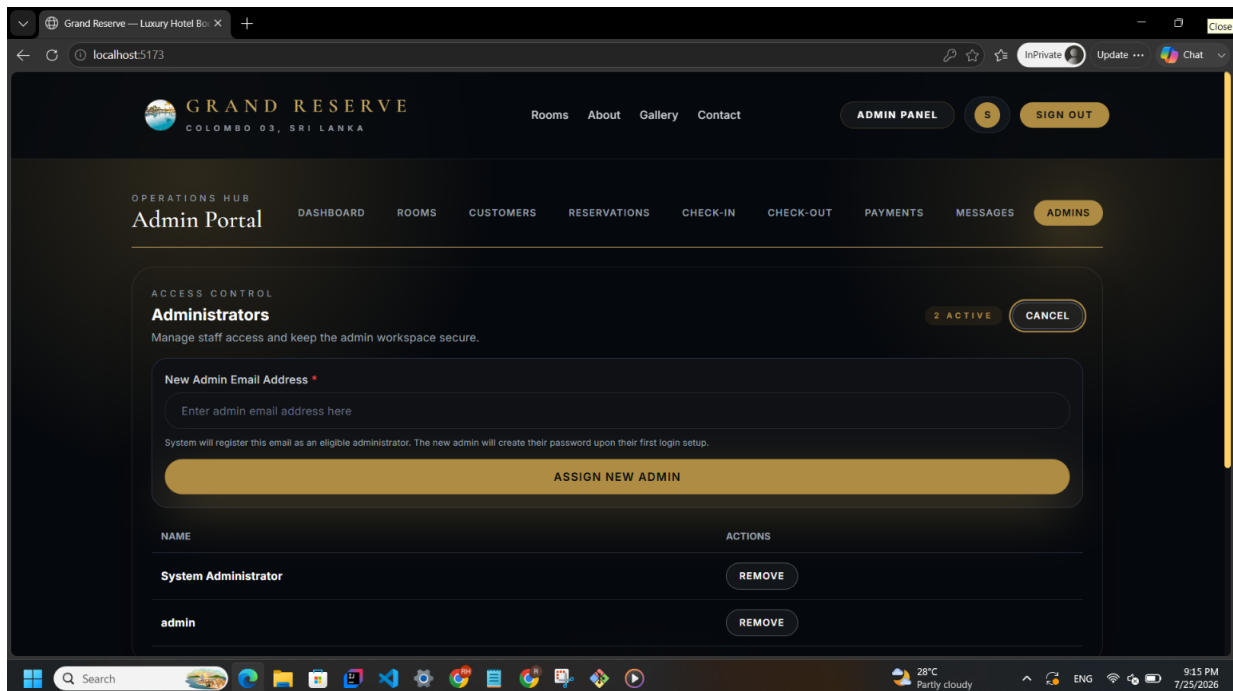
Gallery Section



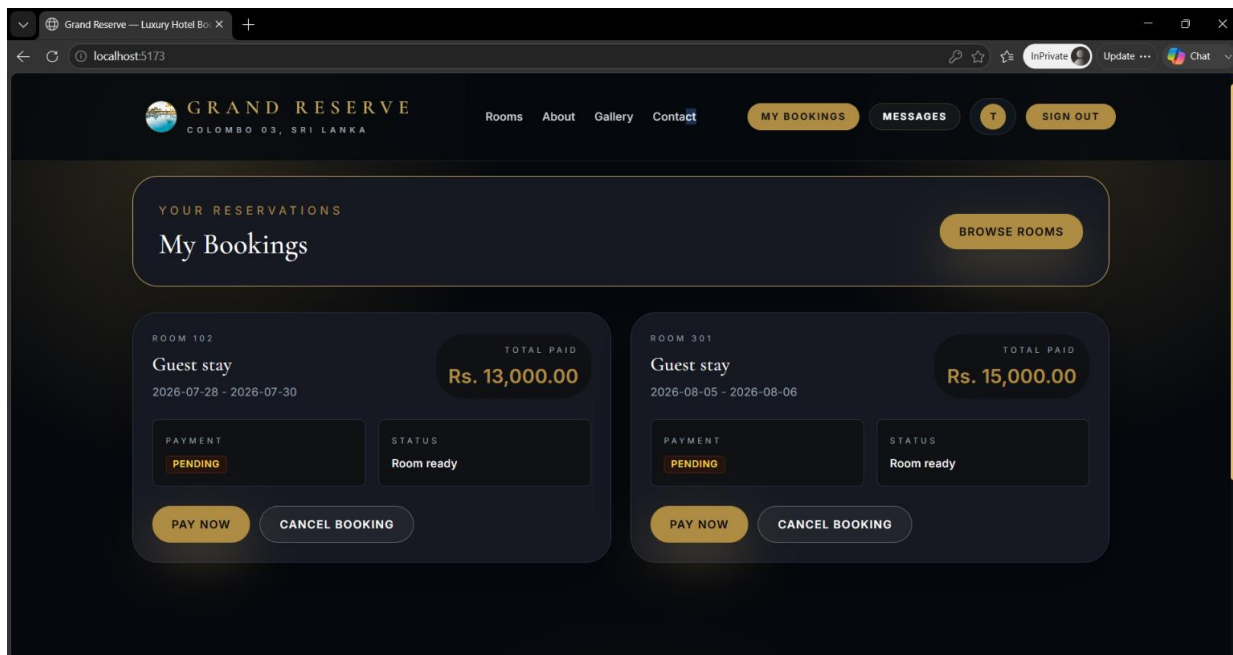
Admin Dashboard



Admin Management Section



My Bookings Section



17. Challenges

- **Preventing double booking**

The reservation service validates room availability and checks for overlapping existing reservations before saving new bookings.

- **Managing reservation lifecycle changes**

Dedicated services were introduced for check-in, check-out, and cancellation to keep state transitions explicit and testable.

- **Securing the application**

JWT authentication and Spring Security were used to protect customer and admin endpoints and enforce role-based access.

- **Handling email delivery gracefully**

The email service is designed to fail safely when SMTP settings are absent, rather than breaking reservation or message workflows.

18. Testing

Test ID	Test Case	Steps	Expected Result	Result
TC-1	Valid login	Submit a valid customer or admin credential pair to /api/auth/login	JWT token is returned and authentication succeeds	Pass
TC-2	Invalid login	Submit an invalid password	Authentication fails and an error response is returned	Pass
TC-3	Room search by date	Enter check-in and check-out dates and request available rooms	Only rooms without conflicting reservations are returned	Pass
TC-4	Reservation creation	Create a reservation for an available room	Reservation is saved and confirmation email is attempted	Pass
TC-5	Overlapping booking prevention	Attempt to book the same room on overlapping dates	Reservation is rejected with an overlap validation error	Pass
TC-6	Room creation by admin	Submit a new room payload through the admin room endpoint	Room is created and becomes available in the room list	Pass
TC-7	Reservation check-in	Invoke the check-in endpoint for a Reserved reservation	Reservation status changes to Checked In and room status updates to Occupied	Pass
TC-8	Reservation check-out	Invoke the check-out endpoint for a	Reservation status changes to	Pass

		Checked In reservation	Checked Out and room status updates to Available	
TC-9	Guest message submission	Submit a guest inquiry through the public messages endpoint	Message is saved and email notification is attempted	Pass
TC-10	Role-based access control	Attempt access to admin-only endpoints as a customer	Access is denied by security configuration	Pass

19. Future Enhancements

- integration with a real payment gateway
- automated invoice generation
- SMS and push notification support
- multi-property or multi-branch hotel support
- mobile-first or native mobile application development
- analytics and reporting enhancements

20. Conclusion

This Hotel Reservation System provides a centralized and efficient digital solution for hotel room reservations. The system is designed to improve booking accuracy, reduce administrative effort, and enhance the overall experience for both customers and administrators.

21. References

1. Spring Boot Documentation. <https://spring.io/projects/spring-boot>
2. Spring Security Documentation. <https://spring.io/projects/spring-security>
3. React Documentation <https://react.dev/>
4. Oracle. Java Platform, Standard Edition Documentation.
<https://docs.oracle.com/en/java/javase/>
5. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
6. JSON Web Token (JWT) Specification. <https://datatracker.ietf.org/doc/html/rfc7519>
7. Java SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/>
8. Project Lombok Documentation. <https://projectlombok.org/>